SEC 8K Product Announcement Analysis

Hieu Nguyen | CAP 5619 | Assignment 2

GitHub repository https://github.com/hingsgn/sec-8k-llm-product-analysis

Purpose and observed results

This project extracts new product announcements from SEC Form 8-K filings into the required five-column CSV. The recorded run analyzed 104 filings from 10 companies and produced 102 reviewed product entries. The retained products come from 31 filings across five issuers; the other 73 filings have no retained product rows. Filing dates span 2023-01-31 through 2025-12-10. Each retained row has a source link and an exact announcement passage; filings without qualifying products remain in the audit records rather than becoming blank output rows.

Data selection and preprocessing

The sample starts with ten Item 2.02 earnings 8-Ks per issuer for NVDA, AMD, INTC, CSCO, CRM, ORCL, IBM, HPE, HPQ and ADBE, selected from 2023–2025. Two older eligible filings each for NVIDIA and AMD raise the total to 104, providing a buffer for the rubric's 100 valid product entries. Technology issuers and earnings releases were chosen because they often summarize product introductions. This is a targeted sample, not a random sample of the S&P 500. The collector resolves tickers with company_tickers.json, retrieves filing histories from the official SEC Submissions JSON API, and follows continuation files when necessary. This replaces the older Atom lookup while using the same SEC source.

The collector downloaded 245 primary reports and attached EX-99 files. BeautifulSoup removes scripts, hidden XBRL content and redundant whitespace. For each filing, the most announcement-rich EX-99 exhibit is selected; the primary report is the fallback. Announcement sentences and brief issuer context form an input passage capped at 10,000 characters. Original cleaned documents remain available for inspection. Requests share a two-per-second limit, local caching and retry backoff.

LLM and named entity recognition

Python calls Qwen2.5-3B-Instruct through llama-cpp-python 0.3.36. The local GGUF file is qwen2.5-3b-instruct-q4_k_m.gguf, using Q4_K_M quantization. Settings are temperature 0, seed 42, an 8,192-token context and up to 900 generated tokens. The pinned publisher revision is 7dabda4d13d513e3e842b20f0d435c732f172cbe. No paid inference API was used.

The prompt performs span-oriented NER and event extraction: it requests the filing company as ORGANIZATION and all newly announced named products as PRODUCT entities in schema-constrained JSON. It excludes financial events, third-party products and existing-product references. The actual responses, token counts, elapsed times and supplied passages are saved. NER offsets refer to these saved passages, making entity labels auditable.

Grounding and output construction

Company names and stock tickers come from SEC issuer metadata. Filing time uses the filing-detail page's Accepted timestamp, interpreted in America/New_York and saved in ISO 8601 form with its UTC offset. It is the SEC filing acceptance time, not the product's original launch date. Product names originate in the LLM response and must map back to source text; only case, spacing and trademark variants are normalized when recovering a source span.

Each accepted product requires a source sentence containing its name and an announcement or availability cue. Descriptions are concise extracts from that evidence, with documented source-based edits for readability. A strict check enforces fewer than 180 characters, including spaces, so 179 is the maximum. Repeated company/product pairs are deduplicated across filings. Coannounced model groups can remain a single entity, which avoids expanding a group into unsupported individual names.

The required products.csv contains exactly company_name, stock_name, filing_time, new_product and product_description, in that order. A separate text file presents the same data with vertical-bar separators. Provenance, NER entities, review decisions, filing metadata and inference logs remain in sidecar files, preserving the required CSV schema.

Challenges and review

A primary 8-K often reports results or refers to an attachment without describing products; following EX-99 exhibits resolves this missing-content problem. Small-model trials also missed product names or returned unrelated entities. The final run uses Qwen 3B with shorter announcement-focused passages. Source matching alone still admits errors: a sentence can announce a partnership, a benchmark result or a customer deployment while naming an existing product. A separate review therefore removes those cases, generic categories, company names and overlapping product references. Original model responses are retained unchanged, while exclusions and description edits are recorded in review_decisions.json.

Validation and limitations

The released CSV passes checks for at least 100 entries, the exact header, complete required fields, timezone-aware timestamps, duplicate pairs, and source evidence for every row. The maximum observed description length is 129 characters. Offline regression checks also reject hallucinated names and ordinary existing-product mentions, and verify trademark recovery and the strict character limit. The notebook was executed against the saved source and inference checkpoints to reproduce the export.

These checks establish formatting and traceability, not a numerical extraction accuracy score. No independent recall benchmark or claim of 100% accuracy is made. The issuer sample, selected exhibit, sentence filtering, passage cap and model omissions can miss valid announcements. The code can scale by adding issuers or widening dates; caching and resumable inference avoid repeating completed work. AI assistance supported implementation, documentation and source-evidence review.

Primary references

SEC EDGAR APIs https://www.sec.gov/search-filings/edgar-application-programming-interfaces

SEC developer guidance https://www.sec.gov/about/developer-resources

Qwen model https://huggingface.co/Qwen/Qwen2.5-3B-Instruct-GGUF

Python runtime documentation https://github.com/abetlen/llama-cpp-python